

Proyecto: Simulación de Persecución Avión–Misil

Documento de Especificación e Interfaces — Grupo 1 (Investigación)

Materia: Modelado y Simulación

Versión: 1.0 — para entrega a los Grupos 2 a 6

Para qué sirve este documento: definir el *concepto del producto*, la *base matemática*, la *arquitectura*, las *dependencias* y, sobre todo, los *contratos de datos (API)*, de modo que los seis grupos puedan trabajar en paralelo **aunque todavía no exista ninguna función implementada**. La estrategia para lograrlo es la de la sección 5.4 (datos de prueba *mock*): el Grupo 2 todavía no terminó el backend, pero los demás grupos construyen contra un resultado de ejemplo con la forma definitiva.

Tabla de contenidos

0. Cómo usar este documento
 1. Visión del producto final
 2. Stack tecnológico y dependencias
 3. Fundamento matemático
 4. Arquitectura del sistema y flujo de datos
 5. Contrato de datos (API central) — **lo más importante**
 6. Responsabilidades por grupo
 7. Especificación de los tres gráficos — **el foco visual**
 8. Pestaña de fundamentos teóricos (Grupo 6)
 9. Plan de integración hasta el jueves
 10. Glosario
-

0. Cómo usar este documento

La idea central del proyecto es una **persecución entre un misil (perseguidor) y un avión (evasor)**. Cada uno es un sistema dinámico: evoluciona según ecuaciones diferenciales, lo que nos permite estudiar estabilidad y estrategias de control. La simulación produce, en cada instante de tiempo, la posición y velocidad de ambos objetos; **esos datos son los que alimentan los tres gráficos**.

Para que todos puedan arrancar hoy mismo:

1. El **Grupo 1** (nosotros) publica este documento y el archivo `types + mockResult` de la sección 5 (los tipos del contrato de datos y un resultado de ejemplo).
2. Los **Grupos 3, 4, 5 y 6** construyen contra ese `mockResult` desde el primer minuto, sin esperar al backend.

3. El **Grupo 2** implementa la función `simulate(config)` respetando exactamente la forma de `SimulationResult`. El día de la integración, se reemplaza el *mock* por la función real y todo encaja.

El contrato de datos (sección 5) es la pieza que sostiene todo. Si todos respetan esa forma, los pedazos encajan sin renegociar nada.

1. Visión del producto final

Una aplicación de una sola página con **dos pestañas**:

- **Pestaña “Simulación”** — el corazón visual. A la izquierda, un panel de controles para cargar las condiciones iniciales y elegir el escenario; a la derecha, los **tres gráficos** dispuestos juntos. Una **barra de reproducción única** (play / pausa / velocidad / barra de desplazamiento temporal) controla **los tres gráficos a la vez**: el avión y el misil se mueven en la grilla 2D, los mismos objetos se mueven en 3D, y un cursor recorre la curva de distancia. Todo sincronizado en el mismo instante de simulación.
- **Pestaña “Fundamentos teóricos”** — las fórmulas en LaTeX, la deducción paso a paso de las ecuaciones y la explicación de por qué el sistema es no lineal, cómo se linealiza y qué dice la estabilidad.

El profesor disfruta lo que se mueve, así que la apuesta principal es la **sincronización de los tres gráficos bajo un solo reproductor** (sección 7.4). Esa es la función que hace que todo se vea vivo.

Flujo de uso típico: El usuario carga posición/velocidad iniciales del avión y del misil, elige la ley de guiado del misil (persecución pura o navegación proporcional) y la maniobra de evasión del avión, presiona “Simular”, y luego reproduce la animación. La app muestra si hubo intercepción, en qué momento y cuál fue la distancia mínima alcanzada.

2. Stack tecnológico y dependencias

El proyecto se construye como una **aplicación web React, con todo corriendo en el navegador** y sin un servidor aparte. Motivo de la decisión: con un plazo al jueves y seis grupos, un esquema cliente/servidor agregaría una capa de red que hay que coordinar y que no aporta nada para esta demo. El cálculo numérico de esta simulación corre sin problema en el navegador, así que no hace falta backend remoto: el “backend lógico” del Grupo 2 es simplemente un módulo de funciones en TypeScript que se ejecuta del lado del cliente.

Capa	Tecnología	Responsable
Framework de UI	React (con Vite como empaquetador)	Grupo 3

Capa	Tecnología	Responsable
Cálculo / backend lógico	Módulo de funciones puras en TypeScript (sin servidor)	Grupo 2
Gráfico 1 (grilla 2D) y Gráfico 3 (distancia)	Plotly.js (vía <code>react-plotly.js</code>)	Grupos 4 y 5
Gráfico 2 (3D)	Three.js (vía <code>@react-three/fiber</code> + <code>@react-three/drei</code>)	Grupos 4 y 5
Renderizado de LaTeX	KaTeX (vía <code>react-katex</code>)	Grupo 6
Tipos compartidos	TypeScript interfaces	Grupo 1

Dependencias a instalar (`npm install`):

```

react react-dom
react-plotly.js plotly.js
three @react-three/fiber @react-three/drei
katex react-katex

```

Más, como dependencias de desarrollo: `vite`, `typescript`, `@types/react`, `@types/three`.

Nota sobre el cálculo numérico: el integrador RK4 (sección 3.5) se implementa a mano en unas pocas líneas. No hace falta una librería de ecuaciones diferenciales. Si quieren operaciones vectoriales más cómodas pueden sumar `mathjs`, pero no es necesario.

3. Fundamento matemático

3.1 Estado y convención de coordenadas

Trabajamos en coordenadas cartesianas. **El mismo modelo sirve para 2D y 3D:** en 2D simplemente fijamos la componente $z = 0$. Esto evita tener dos físicas distintas.

Cada objeto (avión A , misil M) se describe por su **posición** $\mathbf{r} \in \mathbb{R}^3$ y su **velocidad** $\mathbf{v} \in \mathbb{R}^3$:

$$\mathbf{r} = (x, y, z), \quad \mathbf{v} = (v_x, v_y, v_z)$$

El **estado completo del sistema** es la concatenación de los dos:

$$\mathbf{s} = [\mathbf{r}_A, \mathbf{v}_A, \mathbf{r}_M, \mathbf{v}_M] \in \mathbb{R}^{12}$$

La dinámica básica es:

$$\begin{aligned} \dot{\mathbf{r}}_A &= \mathbf{v}_A, & \dot{\mathbf{v}}_A &= \mathbf{a}_A \\ \dot{\mathbf{r}}_M &= \mathbf{v}_M, & \dot{\mathbf{v}}_M &= \mathbf{a}_M \end{aligned}$$

Las aceleraciones $\mathbf{a}_A$ (maniobra de evasión, sección 3.4) y $\mathbf{a}_M$ (ley de guiado, sección 3.3) son lo que hace interesante el problema. En ambos casos las aplicamos **perpendiculares a la velocidad**, lo que cambia la *dirección* del movimiento sin cambiar la *rapidez* — un buen modelo para vehículos que viran a rapidez aproximadamente constante.

Modelo alternativo equivalente (intuición 2D por ángulos). En 2D, en lugar de la velocidad vectorial se puede describir cada objeto por su rapidez v y su ángulo de rumbo θ : $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \frac{a_{\perp}}{v}$ donde $a_{\perp}$ es la aceleración lateral. Esta forma es más intuitiva para explicar “el objeto vira”, y la incluimos en la pestaña teórica. Pero **la forma canónica para implementar es la cartesiana vectorial de arriba**, porque sirve igual en 2D y 3D y da directamente las posiciones que los gráficos necesitan.

3.2 Geometría del encuentro

Estas cantidades describen la relación entre los dos objetos y se calculan en cada instante. Alimentan al guiado y al Gráfico 3.

Vector relativo (del misil hacia el avión), **rango** (distancia) y **versor** de la línea de visión (LOS):

$$\mathbf{r}_{\text{rel}} = \mathbf{r}_A - \mathbf{r}_M, \quad R = \|\mathbf{r}_{\text{rel}}\|, \quad \hat{\mathbf{u}}_{\text{LOS}} = \frac{\mathbf{r}_{\text{rel}}}{R}$$

Velocidad relativa y **velocidad de aproximación** (closing velocity) V_c , que es positiva cuando se acercan:

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_M, \quad \dot{R} = \frac{\mathbf{r}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}}{R}, \quad V_c = -\dot{R}$$

Velocidad de rotación de la línea de visión ω_{LOS} (vector; en 2D solo tiene componente fuera del plano):

$$\omega_{\text{LOS}} = \frac{\mathbf{r}_{\text{rel}} \times \mathbf{v}_{\text{rel}}}{R^2}$$

En 2D, esto se reduce al escalar $\dot{\lambda} = \frac{x_{\text{rel}} \dot{y}_{\text{rel}} - y_{\text{rel}} \dot{x}_{\text{rel}}}{R^2}$, donde $\lambda = \text{atan2}(y_{\text{rel}}, x_{\text{rel}})$ es el ángulo de la LOS.

3.3 Leyes de guiado del misil (cómo se calcula la aceleración del misil)

Ofrecemos dos leyes seleccionables desde la UI. La comparación entre ambas es excelente material para la demo.

(a) Persecución pura (pure pursuit). El misil siempre apunta su velocidad directamente a la posición *actual* del avión. Es la curva de persecución clásica (la del perro que corre hacia su dueño). El comando es girar la velocidad hacia $\hat{\mathbf{u}}_{\text{LOS}}$. Una forma simple y estable de expresarlo como aceleración lateral:

$$\mathbf{a}_M = k (v_M \hat{\mathbf{u}}_{\text{LOS}} - \mathbf{v}_M)_{\perp}$$

donde el subíndice $\perp$ indica la componente perpendicular a $\mathbf{v}_M$ y k es una ganancia de viraje. Produce trayectorias curvas características que “persiguen la cola”.

(b) Navegación proporcional (proportional navigation, PN). Es la ley de guiado real de los misiles y la estrella del proyecto. El misil acelera proporcionalmente a la velocidad de rotación de la línea de visión. Forma vectorial (“true PN”), válida en 2D y 3D:

$$\mathbf{a}_M = N (\boldsymbol{\omega}_{\text{LOS}} \times \mathbf{v}_M)$$

donde N es la **constante de navegación** (típicamente entre 3 y 5). Intuición: si la línea de visión deja de rotar, el misil va en curso de colisión; PN actúa justamente para anular esa rotación. En 2D equivale a la forma escalar conocida $a_M = N V_c \dot{\lambda}$.

3.4 Maniobras de evasión del avión (cómo se calcula la aceleración del avión)

Variar la maniobra del avión da escenarios distintos y vistosos. Proponemos un menú seleccionable:

- **Recta** (sin maniobra): $\mathbf{a}_A = \mathbf{0}$.
- **Viraje constante** (círculo): $\mathbf{a}_A$ de magnitud fija, perpendicular a $\mathbf{v}_A$. Da una trayectoria circular.
- **Serpenteo / weave** (senoidal): aceleración lateral $a_A(t) = A_{\text{max}} \sin(\omega t)$ perpendicular a $\mathbf{v}_A$. Es la maniobra evasiva clásica y se ve muy bien en movimiento.
- **Evasión reactiva**: virar alejándose de la línea de visión al misil (acelerar en la dirección perpendicular que aumenta el rango). Es la más “inteligente” y produce duelos interesantes.

Todas se truncan a una aceleración lateral máxima `maxAccel` (límite físico de maniobra).

3.5 Integración numérica (cómo avanza el tiempo)

Las ecuaciones diferenciales se resuelven numéricamente paso a paso. Recomendamos **Runge–Kutta de 4º orden (RK4)** por su buen equilibrio entre precisión y simplicidad; ofrecemos también Euler como opción didáctica para mostrar (en la demo) por qué un método peor acumula error.

Definimos la función de derivadas del sistema completo $\dot{\mathbf{s}} = f(\mathbf{s}, t)$ (que internamente calcula la geometría 3.2, el guiado 3.3 y la maniobra 3.4). Un paso de RK4 con tamaño Δt :

$$\mathbf{k}_1 = f(\mathbf{s}, t)$$

$$\mathbf{k}_2 = f\left(\mathbf{s} + \frac{\Delta t}{2} \mathbf{k}_1, t + \frac{\Delta t}{2}\right)$$

$$\mathbf{k}_3 = f\left(\mathbf{s} + \frac{\Delta t}{2}\mathbf{k}_2, t + \frac{\Delta t}{2}\right)$$

$$\mathbf{k}_4 = f\left(\mathbf{s} + \Delta t \mathbf{k}_3, t + \Delta t\right)$$

$$\mathbf{s}_{n+1} = \mathbf{s}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

El bucle se detiene cuando $R \leq \text{hitRadius}$ (intercepción) o cuando $t \geq \text{maxTime}$. En cada paso se guarda el estado: ese arreglo de estados a lo largo del tiempo es exactamente el `SimulationResult` de la sección 5.

Referencia de implementación del núcleo (TypeScript, adoptable por el Grupo 2):

```
type Vec3 = [number, number, number];

// Helpers vectoriales (o usar mathjs)
const sub = (a: Vec3, b: Vec3): Vec3 => [a[0]-b[0], a[1]-b[1], a[2]-b[2]];
const add = (a: Vec3, b: Vec3): Vec3 => [a[0]+b[0], a[1]+b[1], a[2]+b[2]];
const scale = (a: Vec3, s: number): Vec3 => [a[0]*s, a[1]*s, a[2]*s];
const dot = (a: Vec3, b: Vec3): number => a[0]*b[0] + a[1]*b[1] + a[2]*b[2];
const cross = (a: Vec3, b: Vec3): Vec3 => [
  a[1]*b[2] - a[2]*b[1],
  a[2]*b[0] - a[0]*b[2],
  a[0]*b[1] - a[1]*b[0],
];
const norm = (a: Vec3): number => Math.sqrt(dot(a, a));

// Aceleración del misil según navegación proporcional (PN)
function missileAccelPN(rA: Vec3, vA: Vec3, rM: Vec3, vM: Vec3, N:
number): Vec3 {
  const rRel = sub(rA, rM);
  const vRel = sub(vA, vM);
  const R2 = dot(rRel, rRel);
  const omegaLOS = scale(cross(rRel, vRel), 1 / R2); // ω_LOS = (r×v)/R²
  return scale(cross(omegaLOS, vM), N); // a_M = N (ω_LOS ×
v_M)
}
```

3.6 Estabilidad y clasificación (lineal / no lineal / homogéneo)

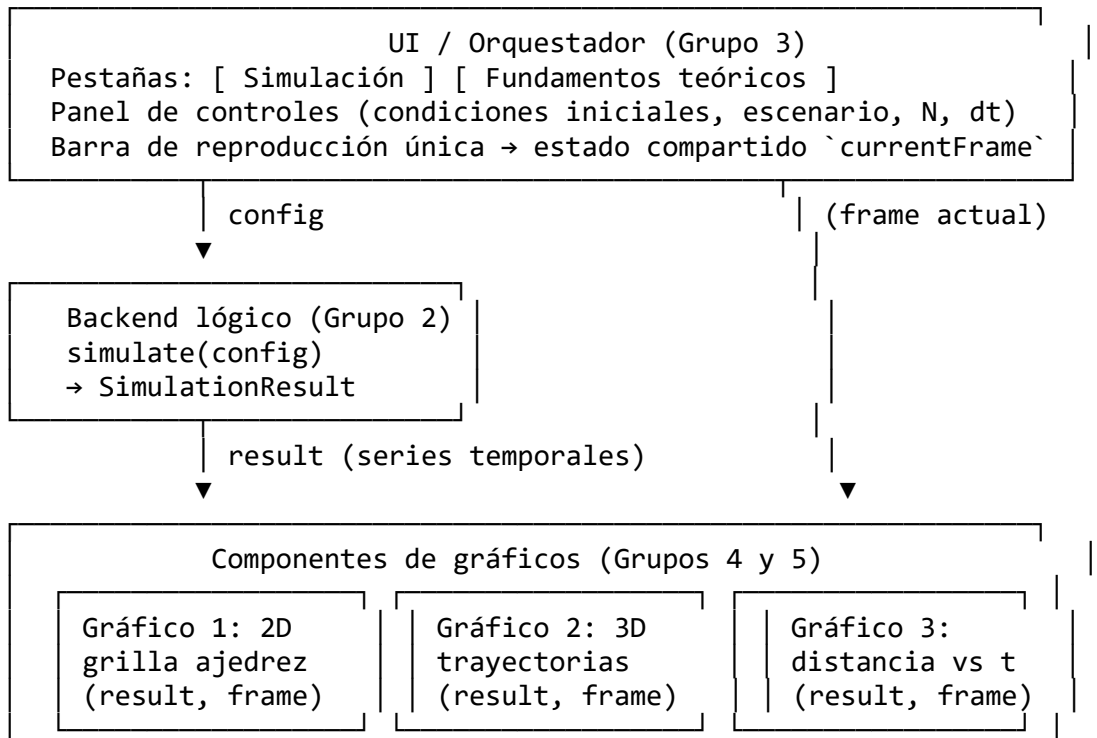
Esta sección conecta el proyecto con lo visto en clase y es el material principal de la pestaña teórica.

- **El sistema completo es NO LINEAL.** La normalización $\mathbf{r}_{\text{rel}}/R$, la división por R^2 y los productos vectoriales hacen que $f(\mathbf{s}, t)$ no sea una función lineal del estado. (Equivalentemente, en la forma por ángulos aparecen $\cos \theta$ y $\sin \theta$.)

- **Linealización.** Si linealizamos las pequeñas desviaciones respecto del *triángulo de colisión nominal* (la geometría en que ambos van directo al impacto), las perturbaciones del estado obedecen un sistema **lineal** $\dot{\mathbf{x}} = A(t)\mathbf{x} + B(t)u$.
- **Sistema HOMOGÉNEO.** Si el avión **no maniobra** (no hay forzado externo), la dinámica del error de puntería (“miss distance”) es un sistema lineal **homogéneo** $\dot{\mathbf{x}} = A(t)\mathbf{x}$, y su respuesta libre es la que decide si el misil intercepta. Cuando el avión sí maniobra, aparece un término de forzado y el sistema pasa a ser **no homogéneo**.
- **Resultado clásico de estabilidad de PN.** La navegación proporcional lleva la velocidad de rotación de la LOS a cero (curso de colisión). Para que la aceleración comandada se mantenga **acotada** hasta el momento de la intercepción (en lugar de dispararse al infinito), se necesita $N > 2$; por eso en la práctica se elige $N = 3$ a 5 . Este es un resultado conocido y citable, y sirve para discutir estabilidad numéricamente: la UI puede dejar variar N y mostrar cómo cambia la curva de distancia y el perfil de aceleración.

Tip para la demo: una función opcional `analyzeStability(config)` puede devolver, para el caso linealizado de coeficientes constantes, los autovalores de A y una etiqueta (“estable” / “inestable”). Es un puente directo entre la materia (sistemas homogéneos y autovalores) y lo que se ve en pantalla.

4. Arquitectura del sistema y flujo de datos



Componentes independientes; reciben result + frame

Pestaña teórica (Grupo 6) – usa KaTeX
Fórmulas, deducción paso a paso, clasificación y estabilidad
(opcional: consume analyzeStability(config) para autovalores)

Reglas de oro de la comunicación:

1. **Una sola dirección para los datos pesados.** El backend produce `SimulationResult`; los gráficos solo lo consumen. Los gráficos **no** calculan física: si un gráfico necesita un número (distancia, velocidad de aproximación), ese número ya viene calculado en el resultado.
2. **La UI es la dueña del tiempo.** Mantiene `currentFrame` (índice del instante actual) y se lo pasa a los tres gráficos. Así los tres muestran siempre el mismo instante. Esto es lo que produce la sincronización (sección 7.4).
3. **Los gráficos son componentes independientes.** Cada uno recibe `result` y `currentFrame` como props y no sabe nada de los otros dos. Se pueden desarrollar y probar por separado contra el `mockResult`.

Estructura de carpetas propuesta (define ownership):

```
/src
  /shared          ← Grupo 1 (este documento lo define)
    types.ts       (SimulationConfig, SimulationResult, enums)
    mockResult.ts  (resultado de ejemplo para trabajar ya)
  /simulation      ← Grupo 2
    index.ts       (export simulate, analyzeStability)
    dynamics.ts    (función de derivadas f)
    guidance.ts    (persecución pura, PN)
    maneuvers.ts   (evasión del avión)
    integrators.ts (rk4Step, eulerStep)
  /ui              ← Grupo 3
    App.tsx        (pestañas + layout)
    Controls.tsx   (formularios de entrada)
    PlaybackBar.tsx (play/pausa/velocidad/scrub → currentFrame)
  /graphs          ← Grupos 4 y 5
    GridView2D.tsx (Gráfico 1)
    Trajectory3D.tsx (Gráfico 2)
    DistancePlot.tsx (Gráfico 3)
  /theory          ← Grupo 6
    TheoryTab.tsx
    formulas.ts    (cadenas LaTeX)
```

5. Contrato de datos (API central) — lo más importante

Esta es la sección que permite el trabajo en paralelo. **Todos los grupos deben respetarla al pie de la letra.** Los tipos viven en `/src/shared/types.ts` y los publica el Grupo 1.

5.1 Entrada: `SimulationConfig`

```
type Vec3 = [number, number, number]; // [x, y, z] en metros (z = 0 para 2D)
```

```
type GuidanceLaw = "pure_pursuit" | "proportional_nav";
```

```
type ManeuverType = "straight" | "constant_turn" | "weave" | "reactive_evade";
```

```
type Integrator = "euler" | "rk4";
```

```
interface SimulationConfig {
  aircraft: {
    position: Vec3;           // posición inicial (m)
    velocity: Vec3;          // velocidad inicial (m/s)
    maneuver: ManeuverType;
    maxAccel: number;        // aceleración lateral máxima (m/s²)
    maneuverParams?: {       // opcional, según la maniobra
      turnRate?: number;     // rad/s, para constant_turn
      weaveAmp?: number;     // m/s², amplitud del serpenteo
      weaveFreq?: number;    // rad/s, frecuencia del serpenteo
    };
  };
  missile: {
    position: Vec3;
    velocity: Vec3;
    guidanceLaw: GuidanceLaw;
    navConstant: number;     // N (solo se usa con proportional_nav), típico 3-5
    maxAccel: number;        // m/s²
  };
  simulation: {
    dt: number;              // paso de tiempo (s), p. ej. 0.02
    maxTime: number;         // tiempo máximo de simulación (s)
    hitRadius: number;       // distancia de intercepción (m)
    integrator: Integrator;
  };
}
```

5.2 Salida: `SimulationResult`

Toda la app gira en torno a este objeto. Cada arreglo está indexado por instante: `time[i]`, `aircraft.position[i]`, `missile.position[i]`, `distance[i]` corresponden todos al mismo paso `i`. Esto hace trivial la sincronización: el `currentFrame` de la UI es simplemente ese índice `i`.

```

interface SimulationResult {
    time: number[];           // [t0, t1, ...] en segundos

    aircraft: {
        position: Vec3[];      // posición en cada instante
        velocity: Vec3[];
        speed: number[];      // |v| en cada instante
    };

    missile: {
        position: Vec3[];
        velocity: Vec3[];
        speed: number[];
        accelCommand: number[]; // magnitud de la aceleración comandada
                                   (m/s2)
    };

    // Cantidades de la geometría del encuentro (alimentan al Gráfico 3)
    distance: number[];        // R = |r_A - r_M| en cada instante
    closingVelocity: number[]; // V_c en cada instante
    losAngle: number[];        // ángulo de la línea de visión (rad),
    útil en 2D

    // Desenlace
    outcome: {
        intercepted: boolean;
        interceptTime: number | null; // s, o null si no hubo intercepción
        minDistance: number;           // distancia mínima alcanzada (m)
        minDistanceTime: number;      // instante de la distancia mínima (s)
    };

    metadata: {
        config: SimulationConfig; // La config con que se generó
        steps: number;           // cantidad de instantes (=
    time.length)
        integrator: Integrator;
    };
}

```

5.3 Funciones que expone cada grupo

Grupo 2 (backend lógico):

```

// Función principal. Determinista: misma config → mismo resultado.
function simulate(config: SimulationConfig): SimulationResult;

// Opcional, para la pestaña teórica (autovalores del caso linealizado).
function analyzeStability(config: SimulationConfig): {
    eigenvalues: { re: number; im: number }[];
    isStable: boolean;
}

```

```
    navConstant: number;
  };
```

Grupos 4 y 5 (gráficos) — cada gráfico es un componente con esta misma forma de props:

```
interface GraphProps {
  result: SimulationResult;
  currentFrame: number; // índice del instante a mostrar (0 ..
  result.steps-1)
}

// <GridView2D    result={...} currentFrame={...} />    (Gráfico 1)
// <Trajectory3D   result={...} currentFrame={...} />    (Gráfico 2)
// <DistancePlot   result={...} currentFrame={...} />    (Gráfico 3)
```

Grupo 3 (UI) posee el estado config y currentFrame, llama a simulate(config), guarda el result y se lo reparte a los tres gráficos junto con el currentFrame. La PlaybackBar incrementa currentFrame con un temporizador (requestAnimationFrame).

Grupo 6 (teoría) consume opcionalmente analyzeStability(config) y renderiza las cadenas LaTeX de formulas.ts.

5.4 Datos de prueba (mockResult) para empezar HOY

El Grupo 1 publica este objeto en /src/shared/mockResult.ts. Permite que los Grupos 3, 4, 5 y 6 construyan **sin esperar al Grupo 2**. (Es un ejemplo recortado a pocos pasos; el real tendrá cientos.)

```
export const mockResult: SimulationResult = {
  time: [0.0, 0.1, 0.2, 0.3, 0.4],
  aircraft: {
    position: [[0,0,0], [20,2,0], [40,5,0], [60,9,0], [80,14,0]],
    velocity: [[200,15,0],[200,25,0],[200,38,0],[200,48,0],[200,55,0]],
    speed: [200.6, 201.6, 203.6, 205.7, 207.4],
  },
  missile: {
    position: [[0,500,0], [55,480,0], [110,455,0], [165,425,0],
    [220,390,0]],
    velocity: [[550,-200,0],[555,-245,0],[558,-295,0],[560,-340,0],[562,-
    380,0]],
    speed: [585, 606, 632, 655, 678],
    accelCommand: [0, 120, 180, 210, 195],
  },
  distance: [500.0, 480.9, 453.5, 419.5, 380.3],
  closingVelocity: [200, 210, 268, 330, 392],
  losAngle: [1.5708, 1.4001, 1.2793, 1.1607, 1.0517],
  outcome: {
    intercepted: false,
    interceptTime: null,
  },
}
```

```

    minDistance: 380.3,
    minDistanceTime: 0.4,
  },
  metadata: {
    config: { /* ...una SimulationConfig de ejemplo... */ } as any,
    steps: 5,
    integrator: "rk4",
  },
};

```

Los números del *mock* son ilustrativos para tener algo con que dibujar; no salen de una corrida real. Lo que importa es que **la forma** coincide exactamente con `SimulationResult`.

6. Responsabilidades por grupo

Grupo 1 — Investigación (nosotros). Entregar este documento, `types.ts` y `mockResult.ts`. Mantener el contrato de datos y resolver dudas de interfaz que surjan durante la integración. Somos el árbitro si dos grupos discrepan sobre la forma de los datos.

Grupo 2 — Backend lógico. Implementar `simulate(config)` respetando `SimulationResult` exactamente: la función de derivadas $\dot{f}$ (geometría 3.2 + guiado 3.3 + maniobra 3.4), el integrador RK4/Euler (3.5), la detección de intercepción y el cálculo de las cantidades del encuentro (distancia, V_c , ángulo LOS) en cada paso. Opcionalmente `analyzeStability`. Entregable temprano útil: una corrida real exportada como JSON para reemplazar el *mock*.

Grupo 3 — UI / orquestador. Cáscara con dos pestañas, formularios de carga de condiciones iniciales y escenario, validación de entradas, llamada a `simulate`, almacenamiento del `result`, y la **barra de reproducción** que maneja `currentFrame` (play, pausa, velocidad $\times 0.5/\times 1/\times 2$, barra de desplazamiento). Reparte `result` + `currentFrame` a los tres gráficos. Es el “director de orquesta”.

Grupos 4 y 5 — Gráficos (foco del proyecto). Los tres componentes de la sección 7. Sugerencia de reparto: un grupo toma el Gráfico 2 (3D, el más exigente técnicamente) y el otro toma los Gráficos 1 y 3 (ambos 2D, comparten herramienta Plotly). Coordinen una **paleta común** (mismo color para el avión y mismo color para el misil en los tres gráficos) para que el profesor los lea como una sola historia.

Grupo 6 — Fundamentos teóricos. Pestaña con las fórmulas de la sección 3 en LaTeX (KaTeX), la deducción paso a paso de las ecuaciones de movimiento y del guiado, y la explicación de la clasificación lineal/no lineal/homogéneo y la estabilidad (3.6). Opcionalmente mostrar los autovalores de `analyzeStability`. Las cadenas LaTeX listas para usar están en la sección 8.

7. Especificación de los tres gráficos (el foco visual)

Los tres reciben siempre `result` y `currentFrame`. Convención: dibujan el **rastro completo hasta `currentFrame`** y un **marcador en la posición actual** `currentFrame`. Así, al avanzar el frame, los rastros crecen y los marcadores se mueven.

7.1 Gráfico 1 — Grilla 2D tipo ajedrez (vista cenital)

- **Qué muestra:** vista desde arriba (plano $x-y$, se ignora z o se codifica con tamaño/opacidad del marcador). Fondo de **grilla tipo tablero de ajedrez** (cuadros alternados claros/oscuros) para dar sensación de escala y movimiento.
- **Elementos:** marcador del **avión** (p. ej. triángulo orientado según su rumbo) y del **misil** (p. ej. rombo o flecha), con su **rastro** (la trayectoria recorrida hasta el frame actual). Una **línea de visión** punteada uniendo ambos en el instante actual es un detalle muy vistoso y didáctico (muestra cómo gira —o no— la LOS).
- **Herramienta:** Plotly.js. El tablero se puede dibujar con shapes (rectángulos alternados) o una capa de fondo; los rastros con scatter modo líneas; los marcadores con scatter modo markers usando solo el punto `currentFrame`.
- **Datos que usa:** `result.aircraft.position`, `result.missile.position` (componentes x, y), recortados a `[0 .. currentFrame]`.

7.2 Gráfico 2 — Trayectoria 3D (la estrella)

- **Qué muestra:** escena 3D con el avión y el misil como objetos tridimensionales (basta un **cono/flecha** orientado según la velocidad para cada uno; si sobra tiempo, un modelo simple) recorriendo sus **trayectorias 3D** dibujadas como líneas/tubos. Cámara orbitable por el usuario.
- **Animación:** en cada `currentFrame`, los objetos se ubican en `position[currentFrame]` y se orientan según `velocity[currentFrame]`; los tubos de trayectoria crecen hasta ese índice.
- **Herramienta recomendada:** **Three.js** (con `@react-three/fiber`) para el mejor efecto (iluminación, orientación de los objetos, cámara). **Alternativa más simple:** un `scatter3d` de Plotly con `mode: "lines+markers"`, que da 3D interactivo con mucho menos código (buen plan B si el tiempo aprieta).
- **Detalle lindo:** una grilla en el “piso” ($z = 0$) y la proyección de las sombras de cada objeto sobre ese piso ayuda a leer la altura.
- **Datos que usa:** `result.aircraft.position`, `result.missile.position`, `velocity` (para orientar), recortados a `currentFrame`.

7.3 Gráfico 3 — Distancia vs tiempo (la que va a encantar al profe)

- **Qué muestra:** curva 2D con tiempo en el eje horizontal y **distancia** R entre avión y misil en el vertical (`result.time` vs `result.distance`). Es la lectura más clara de “qué tan cerca está de interceptar”.
- **Cursor sincronizado:** una **línea vertical** en `time[currentFrame]` que barre la curva a medida que avanza la reproducción. Ese cursor moviéndose en

sincronía con los otros dos gráficos es justo el tipo de movimiento que buscamos.

- **Anotaciones:** marcar el punto de **distancia mínima** (`outcome.minDistance` en `outcome.minDistanceTime`) y, si hubo intercepción, una marca en `outcome.interceptTime`. Opcional: una segunda curva con la **velocidad de aproximación** V_c en un eje secundario.
- **Herramienta:** Plotly.js (scatter líneas + una shape de línea vertical para el cursor).
- **Datos que usa:** `result.time`, `result.distance`, `result.closingVelocity`, `result.outcome`.

7.4 Sincronización: el reproductor único (la función clave)

Esto es lo que hace que todo “esté vivo” y es la pieza que recomendamos cuidar más.

- La UI (Grupo 3) mantiene **un solo** estado `currentFrame` (un entero de 0 a `result.steps - 1`).
- La `PlaybackBar` lo incrementa con `requestAnimationFrame`, a un ritmo modulado por la velocidad elegida ($\times 0.5$, $\times 1$, $\times 2$). Play arranca el bucle, Pausa lo detiene, y la barra de desplazamiento lo fija manualmente.
- Ese **mismo** `currentFrame` se pasa como prop a los tres gráficos. Como los tres arreglos están alineados por índice (sección 5.2), los tres muestran exactamente el mismo instante: el avión y el misil aparecen en la misma posición en 2D y en 3D, y el cursor de distancia marca ese mismo tiempo. **Cero lógica de sincronización adicional:** alcanza con compartir un número.

// Esqueleto del bucle de reproducción (Grupo 3)

```
const [currentFrame, setCurrentFrame] = useState(0);
const [playing, setPlaying] = useState(false);
const [speed, setSpeed] = useState(1);

useEffect(() => {
  if (!playing) return;
  let raf: number;
  let last = performance.now();
  const tick = (now: number) => {
    const dtReal = (now - last) / 1000;
    last = now;
    setCurrentFrame((f) => {
      const next = f + speed * dtReal /
result.metadata.config.simulation.dt;
      return Math.min(Math.floor(next), result.steps - 1);
    });
    raf = requestAnimationFrame(tick);
  };
  raf = requestAnimationFrame(tick);
  return () => cancelAnimationFrame(raf);
}, [playing, speed, result]);
```

8. Pestaña de fundamentos teóricos (Grupo 6)

Estructura sugerida de la pestaña, con las cadenas LaTeX listas para pegar en KaTeX (react-katex). El objetivo es que el profesor vea la cadena completa: planteo → ecuaciones → clasificación → estabilidad.

8.1 Planteo del problema. Dos sistemas dinámicos en persecución; presentar el estado $\mathbf{s} \in \mathbb{R}^{12}$ y las ecuaciones de la sección 3.1.

8.2 Ecuaciones de movimiento (cadenas LaTeX):

```
\dot{\mathbf{r}}_A = \mathbf{v}_A, \quad \dot{\mathbf{v}}_A = \mathbf{a}_A  
\dot{\mathbf{r}}_M = \mathbf{v}_M, \quad \dot{\mathbf{v}}_M = \mathbf{a}_M
```

Forma por ángulos (intuición 2D):

```
\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \frac{a_{\perp}}{v}
```

8.3 Geometría del encuentro:

```
\mathbf{r}_{\text{rel}} = \mathbf{r}_A - \mathbf{r}_M, \quad R = |\mathbf{r}_{\text{rel}}|  
V_c = -\dot{R} = -\frac{\mathbf{r}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}}{R}  
\boldsymbol{\omega}_{\text{LOS}} = \frac{\mathbf{r}_{\text{rel}} \times \mathbf{v}_{\text{rel}}}{R^2}
```

8.4 Leyes de guiado:

```
\text{Navegación proporcional:} \quad \mathbf{a}_M = N, (\boldsymbol{\omega}_{\text{LOS}} \times \mathbf{v}_M)  
\text{En 2D:} \quad a_M = N, V_c, \dot{\lambda}
```

8.5 Método numérico (RK4): las cinco líneas de la sección 3.5.

8.6 Clasificación y estabilidad. Texto explicativo: - Por qué el sistema es **no lineal** (normalización, división por R^2 , productos cruzados; o $\sin / \cos$). - Cómo se **lineariza** alrededor del curso de colisión: $\dot{\mathbf{x}} = A(t)\mathbf{x} + B(t)\mathbf{u}$. - Por qué, sin maniobra del avión, queda un sistema **homogéneo** $\dot{\mathbf{x}} = A(t)\mathbf{x}$ (y por qué con maniobra aparece el forzado → no homogéneo). - El resultado clásico: PN anula la rotación de la LOS; la aceleración se mantiene acotada hasta el impacto si $N > 2$ (por eso $N \in [3, 5]$). - Opcional: mostrar los autovalores de A que devuelve `analyzeStability` y la conclusión de estabilidad. Esto enlaza directo con los sistemas homogéneos y los autovalores de la materia.

8.7 Paso a paso de una corrida (opcional, muy valorado): tomar la config por defecto y mostrar, para los primeros 2–3 pasos, los números de $\mathbf{r}_{\text{rel}}$, R , ω_{LOS} , $\mathbf{a}_M$ y la actualización RK4. Da concreción a las fórmulas.

9. Plan de integración hasta el jueves

Orden sugerido para que nada se bloquee:

1. **Hoy (Grupo 1):** publicar este documento + `types.ts` + `mockResult.ts`. A partir de acá, **nadie está bloqueado**.
2. **Arranque inmediato y en paralelo:**
 - Grupo 2 implementa `simulate` contra los tipos.
 - Grupos 3, 4, 5, 6 construyen contra `mockResult`.
3. **Primer hito (cuando el Grupo 2 tenga una corrida):** exportar un `result` real a JSON y reemplazar el `mock`. Verificar que los tres gráficos lo dibujan sin tocar nada (esa es la prueba de que el contrato se respetó).
4. **Integración:** conectar la `PlaybackBar` con los tres gráficos y validar la sincronización (sección 7.4).
5. **Pulido visual:** paleta común avión/misil en los tres gráficos, anotaciones de distancia mínima e intercepción, tablero del Gráfico 1, sombras del Gráfico 2.
6. **Ensayo de la demo:** preparar 2–3 escenarios guardados que se vean bien (p. ej. una intercepción limpia con PN y $N = 4$, y una evasión exitosa con serpenteo), para mostrarlos en vivo sin cargar datos a mano.

Criterio de “listo”: se carga un escenario, se presiona `Simular`, y al reproducir se ven el avión y el misil moviéndose sincronizados en 2D y en 3D mientras el cursor recorre la curva de distancia, terminando en intercepción o en la distancia mínima.

10. Glosario

- **LOS (Line of Sight, línea de visión):** recta imaginaria que une al misil con el avión. Su rotación es la clave del guiado.
- **Rango (R):** distancia entre ambos objetos. Es la variable del Gráfico 3.
- **Velocidad de aproximación (V_c):** cuán rápido se acortan; positiva si se acercan.
- **Navegación proporcional (PN):** ley de guiado que acelera el misil proporcionalmente a la rotación de la LOS; constante N .
- **Persecución pura:** el misil apunta siempre a la posición actual del avión.
- **RK4:** método de Runge–Kutta de 4º orden para integrar las ecuaciones diferenciales paso a paso.
- **Sistema homogéneo:** $\dot{\mathbf{x}} = A\mathbf{x}$, sin término de forzado; aquí, la dinámica del error cuando el avión no maniobra.

- **currentFrame:** índice del instante de simulación que se está mostrando; el número que sincroniza los tres gráficos.

Documento elaborado por el Grupo 1 (Investigación). Ante cualquier duda sobre la forma de los datos o las interfaces, consultar con el Grupo 1 antes de implementar para no romper el contrato.